Chapitre 3 — Types d'API et Formats de Données

Notes essentielles — Master Ingénierie Informatique

Objectifs de la séance

- Classer une API selon sa portée d'accès
- Distinguer type d'API et style architectural
- Lire et écrire du JSON valide
- Lire et écrire du XML valide
- Comparer JSON, XML, HTML, YAML
- Maîtriser la négociation de contenu HTTP

Plan de la séance (4h)

- Les 4 types d'API par portée d'accès (35 min)
- Types vs styles architecturaux (25 min)
- JSON en profondeur (25 min)
- Pause (15 min)
- XML en profondeur (30 min)
- HTML et YAML (20 min)
- Négociation de contenu HTTP (35 min)
- Exercices dirigés (35 min)

API interne / privée

- Usage exclusivement interne à une organisation
- Exemple : connecter le système RH au système de paie
- Souvent protégée par l'infrastructure réseau (VPN, IP allow-list)
- Documentation parfois minimale — attention, ça coûte cher à terme

API externe / publique

- Ouverte à des tiers, parfois au grand public
- Exemple : l'API Google Maps
- Documentation publique rigoureuse indispensable
- Versionnage strict (/v1/, /v2/), clés d'API, quotas

API partenaire

- Restreinte à un ensemble défini de partenaires
- Exemple : synchronisation avec un transporteur logistique
- Accès conditionné par un accord contractuel (SLA)
- Authentification renforcée (certificats, clés dédiées)

API composite

- Combine plusieurs appels en un seul côté client
- Optimise les performances (moins d'allers-retours réseau)
- N'est pas définie par la portée d'accès, mais par l'agrégation
- Peut elle-même être interne, externe ou partenaire

Types vs styles architecturaux

- Type d'API : qui peut l'appeler ? (Interne, Externe, Partenaire, Composite)
- Style d'API : comment est-elle construite ? (REST, SOAP, GraphQL, gRPC)
- Ces deux classifications sont orthogonales
- Une API REST peut être interne, externe ou partenaire — et inversement

JSON — définition

- JavaScript Object Notation
- Format texte léger d'échange de données
- Indépendant de tout langage de programmation
- Standard RFC 8259

JSON — les 6 types de valeurs

- String : entre guillemets doubles — "Jean"
- Number : entier ou décimal, sans guillemets — 30
- Boolean : true / false, sans guillemets
- Null : null, sans guillemets
- Object : paires clé/valeur entre { }
- Array : liste ordonnée entre [ ]

JSON — règles de rigueur

- Clés toujours entre guillemets doubles
- Pas de virgule finale après le dernier élément
- Pas de commentaires en JSON
- Une seule valeur racine (objet ou tableau)

Pourquoi JSON domine aujourd'hui

- Poids réduit — payloads plus légers
- Correspondance directe avec les structures de données des langages
- Lisibilité humaine immédiate
- Parsing natif en JavaScript (JSON.parse / JSON.stringify)

XML — définition

- eXtensible Markup Language
- Langage de balisage extensible
- Les balises sont définies par l'auteur du document
- Structure hiérarchique de nœuds

XML — vocabulaire structurel

- Nœud : unité de base, balise ouvrante/fermante
- Racine : l'unique nœud de plus haut niveau
- Élément : balise + contenu — <nom>Jean</nom>
- Attribut : info dans la balise — <personne id="42">
- Parent / enfant : relation hiérarchique

XML — validation de structure

- DTD (Document Type Definition)
- XSD (XML Schema Definition)
- Décrit précisément la structure autorisée d'un document
- JSON n'a pas d'équivalent natif aussi répandu
- Argument fort en faveur de XML (finance, santé, SOAP)

JSON vs XML

- Verbosité : faible (JSON) vs élevée (XML)
- Attributs : n'existe pas en JSON vs existe en XML
- Listes : type natif en JSON vs répétition d'éléments en XML
- Commentaires : non supportés en JSON vs supportés en XML
- Validation de schéma native : non (JSON) vs oui (XML)

HTML comme format de réponse

- Pas conçu comme un format d'échange programme-à-programme
- Cas particulier : aperçu HTML généré par une API
- Mauvaise pratique si le consommateur visé est un programme

YAML

- YAML Ain't Markup Language
- Centré sur la lisibilité humaine
- Indentation significative (comme Python)
- Sur-ensemble de JSON
- Usage dominant : fichiers de configuration (Docker, Kubernetes, CI/CD)
- Rarement utilisé comme format d'échange API

Au-delà du texte : Protocol Buffers

- Tous les formats vus sont textuels (lisibles par un humain)
- Protocol Buffers (protobuf) : format binaire, utilisé par gRPC
- Beaucoup plus compact et rapide que JSON/XML
- Illisible directement, nécessite un schéma .proto
- Détaillé au Chapitre 5

Négociation de contenu HTTP

- Un même serveur peut répondre en JSON à un client, en XML à un autre
- Content-Type : indique le format du corps envoyé
- Accept : indique les formats de réponse tolérés par le client
- Ces deux en-têtes répondent à des questions différentes

Types MIME essentiels

- JSON : application/json
- XML : application/xml
- HTML : text/html
- YAML : application/x-yaml
- Formulaire web : application/x-www-form-urlencoded

Erreurs de négociation

- 415 Unsupported Media Type : le serveur ne comprend pas le Content-Type envoyé
- 406 Not Acceptable : le serveur ne peut produire aucun format demandé dans Accept
- 415 concerne ce que le client envoie
- 406 concerne ce que le client demande en retour

Exercices de la séance

- Classifier des scénarios par type d'accès
- Débusquer des erreurs de syntaxe JSON
- Traduire du JSON vers XML
- Écrire des en-têtes de négociation de contenu
- Diagnostiquer un code d'erreur

Synthèse

- 4 types d'API par portée d'accès : Interne, Externe, Partenaire, Composite
- Type d'API et style architectural sont orthogonaux
- JSON : léger, dominant avec REST/GraphQL
- XML : verbeux mais validable (DTD/XSD), dominant avec SOAP
- La négociation de contenu HTTP permet plusieurs formats sur un même serveur

Prochain chapitre

- Chapitre 4 : REST et SOAP
- Le premier style architectural concret
- Comment structurer une API cohérente au-dessus de HTTP